

Master SQL — Complete Professional Guide

SQL Fundamentals to Advanced Concepts with Real-World Examples

Table of Contents

1. Introduction to SQL
2. Understanding Databases
3. SQL Categories Overview
4. Basic SQL Syntax
5. SELECT Statement Deep Dive
6. Filtering Data with WHERE
7. Sorting Data with ORDER BY
8. GROUP BY and Aggregation
9. HAVING Clause
10. DDL — Data Definition Language
11. DML — Data Manipulation Language
12. DCL — Data Control Language
13. TCL — Transaction Control Language
14. SQL Constraints
15. Primary Keys and Foreign Keys
16. Joins Deep Dive
17. INNER JOIN
18. LEFT JOIN
19. RIGHT JOIN
20. FULL OUTER JOIN
21. CROSS JOIN
22. SELF JOIN
23. Subqueries Deep Dive
24. Correlated Subqueries
25. EXISTS and NOT EXISTS
26. Views in SQL
27. Indexes in SQL
28. Common Table Expressions (CTEs)
29. Recursive CTEs
30. Window Functions Deep Dive
31. Ranking Functions
32. Aggregate Window Functions
33. Running Totals and Moving Averages
34. CASE Statements
35. Set Operators
36. Stored Procedures and Functions
37. Transactions and ACID Properties
38. SQL Performance Optimization
39. Real-World SQL Scenarios

- 40. SQL Best Practices
 - 41. Common SQL Mistakes
 - 42. SQL Interview-Level Concepts
 - 43. End-to-End SQL Project Example
 - 44. Final Summary
-

1. Introduction to SQL

SQL (Structured Query Language) is the standard language used to:

- store data,
- retrieve data,
- manipulate data,
- manage databases,
- analyze information.

SQL is used in:

- Data Engineering
- Data Analytics
- Backend Development
- Machine Learning Pipelines
- Business Intelligence
- Enterprise Applications

Popular databases include:

Database	Type
MySQL	Relational
PostgreSQL	Relational
SQL Server	Enterprise
Oracle	Enterprise
Snowflake	Cloud Data Warehouse
BigQuery	Serverless Analytics
Redshift	AWS Data Warehouse

2. Understanding Databases

A database stores structured information.

Example:

Customers Table

customer_id	customer_name	city
101	John	Dallas
102	Alice	Atlanta

Each row represents a record.

Each column represents an attribute.

3. SQL Categories Overview

SQL commands are divided into categories.

Category	Purpose
DDL	Define structures
DML	Manipulate data
DCL	Control permissions
TCL	Manage transactions
Query Language	Retrieve data

4. Basic SQL Syntax

The SELECT statement retrieves data.

Example

```
SELECT *  
FROM customers;
```

SQL Syntax Structure

```
SELECT column_name  
FROM table_name  
WHERE condition  
GROUP BY column_name
```

```
HAVING condition  
ORDER BY column_name;
```

5. SELECT Statement Deep Dive

Used to retrieve data.

Select Specific Columns

```
SELECT customer_name, city  
FROM customers;
```

Select All Columns

```
SELECT *  
FROM customers;
```

Remove Duplicates with DISTINCT

```
SELECT DISTINCT city  
FROM customers;
```

Aliasing Columns

```
SELECT customer_name AS name  
FROM customers;
```

6. Filtering Data with WHERE

Filters rows based on conditions.

Example

```
SELECT *  
FROM customers  
WHERE city = 'Dallas';
```

Common Operators

Operator	Description
=	Equal
!=	Not Equal
>	Greater Than
<	Less Than
BETWEEN	Range
IN	Match values
LIKE	Pattern matching

BETWEEN Example

```
SELECT *  
FROM orders  
WHERE amount BETWEEN 100 AND 500;
```

IN Example

```
SELECT *  
FROM customers  
WHERE city IN ('Dallas', 'Austin');
```

LIKE Example

```
SELECT *  
FROM customers  
WHERE customer_name LIKE 'J%';
```

7. Sorting Data with ORDER BY

Used to sort query results.

Ascending Order

```
SELECT *  
FROM orders  
ORDER BY amount ASC;
```

Descending Order

```
SELECT *  
FROM orders  
ORDER BY amount DESC;
```

8. GROUP BY and Aggregation

GROUP BY groups rows for aggregation.

Aggregate Functions

Function	Description
COUNT()	Count rows
SUM()	Total value
AVG()	Average
MAX()	Maximum

Function	Description
MIN()	Minimum

Example — Revenue by City

```
SELECT city,  
       SUM(amount) AS total_revenue  
FROM orders  
GROUP BY city;
```

9. HAVING Clause

HAVING filters grouped results.

Example

```
SELECT city,  
       SUM(amount) AS revenue  
FROM orders  
GROUP BY city  
HAVING SUM(amount) > 1000;
```

10. DDL — Data Definition Language

DDL defines database structures.

Commands:

- CREATE
 - ALTER
 - DROP
 - TRUNCATE
-

CREATE TABLE

```
CREATE TABLE customers (  
    customer_id INT,  
    customer_name VARCHAR(100),  
    city VARCHAR(50)  
);
```

ALTER TABLE

```
ALTER TABLE customers  
ADD email VARCHAR(100);
```

DROP TABLE

```
DROP TABLE customers;
```

TRUNCATE TABLE

```
TRUNCATE TABLE customers;
```

11. DML — Data Manipulation Language

Used to manipulate data.

Commands:

- INSERT
 - UPDATE
 - DELETE
-

INSERT

```
INSERT INTO customers  
VALUES (101, 'John', 'Dallas');
```

UPDATE

```
UPDATE customers  
SET city = 'Austin'  
WHERE customer_id = 101;
```

DELETE

```
DELETE FROM customers  
WHERE customer_id = 101;
```

12. DCL — Data Control Language

Controls access and permissions.

Commands:

- GRANT
 - REVOKE
-

GRANT Example

```
GRANT SELECT ON customers TO analyst;
```

REVOKE Example

```
REVOKE SELECT ON customers FROM analyst;
```

13. TCL — Transaction Control Language

Manages transactions.

Commands:

- COMMIT
- ROLLBACK
- SAVEPOINT

Transaction Example

```
BEGIN;  
  
UPDATE accounts  
SET balance = balance - 100  
WHERE account_id = 1;  
  
UPDATE accounts  
SET balance = balance + 100  
WHERE account_id = 2;  
  
COMMIT;
```

14. SQL Constraints

Constraints enforce data integrity.

Constraint	Purpose
PRIMARY KEY	Unique identifier
FOREIGN KEY	Relationship
UNIQUE	No duplicates
NOT NULL	Required value
CHECK	Validation
DEFAULT	Default value

Example

```
CREATE TABLE employees (  
    employee_id INT PRIMARY KEY,  
    salary DECIMAL(10,2) CHECK (salary > 0)  
);
```

15. Primary Keys and Foreign Keys

Primary Key

Uniquely identifies a row.

Foreign Key

Creates relationships between tables.

Example

```
CREATE TABLE orders (  
    order_id INT PRIMARY KEY,  
    customer_id INT,  
    FOREIGN KEY (customer_id)  
        REFERENCES customers(customer_id)  
);
```

16. Joins Deep Dive

Joins combine data from multiple tables.

Example Tables

Customers

customer_id	customer_name
101	John
102	Alice

Orders

order_id	customer_id	amount
1	101	500
2	102	300

17. INNER JOIN

Returns matching rows.

```
SELECT c.customer_name,  
       o.amount  
FROM customers c  
INNER JOIN orders o  
ON c.customer_id = o.customer_id;
```

INNER JOIN Diagram

Customers $\cap$ Orders

18. LEFT JOIN

Returns all rows from left table.

```
SELECT c.customer_name,  
       o.amount  
FROM customers c
```

```
LEFT JOIN orders o
ON c.customer_id = o.customer_id;
```

LEFT JOIN Diagram

Customers $\leftarrow$ Orders

19. RIGHT JOIN

Returns all rows from right table.

```
SELECT c.customer_name,
       o.amount
FROM customers c
RIGHT JOIN orders o
ON c.customer_id = o.customer_id;
```

20. FULL OUTER JOIN

Returns all matching and non-matching rows.

```
SELECT *
FROM customers c
FULL OUTER JOIN orders o
ON c.customer_id = o.customer_id;
```

21. CROSS JOIN

Returns Cartesian product.

```
SELECT *
FROM colors
CROSS JOIN sizes;
```

22. SELF JOIN

Joins a table with itself.

Example:

```
SELECT e.employee_name,  
       m.employee_name AS manager  
FROM employees e  
JOIN employees m  
ON e.manager_id = m.employee_id;
```

23. Subqueries Deep Dive

Subqueries are queries inside another query.

Example

```
SELECT customer_name  
FROM customers  
WHERE customer_id IN (  
    SELECT customer_id  
    FROM orders  
    WHERE amount > 500  
);
```

Types of Subqueries

Type	Description
Scalar	Returns one value
Multi-row	Returns multiple rows
Correlated	Depends on outer query

24. Correlated Subqueries

Executed once per outer row.

Example

```
SELECT customer_name
FROM customers c
WHERE EXISTS (
    SELECT 1
    FROM orders o
    WHERE o.customer_id = c.customer_id
);
```

25. EXISTS and NOT EXISTS

EXISTS

Returns TRUE if subquery returns rows.

```
SELECT customer_name
FROM customers c
WHERE EXISTS (
    SELECT 1
    FROM orders o
    WHERE o.customer_id = c.customer_id
);
```

NOT EXISTS

```
SELECT customer_name
FROM customers c
WHERE NOT EXISTS (
    SELECT 1
    FROM orders o
    WHERE o.customer_id = c.customer_id
);
```

26. Views in SQL

Views are virtual tables.

Create View

```
CREATE VIEW customer_orders AS
SELECT c.customer_name,
       o.amount
FROM customers c
JOIN orders o
ON c.customer_id = o.customer_id;
```

Advantages of Views

- reusable queries,
- simplified reporting,
- improved security,
- abstraction layer.

Materialized Views

Stores query results physically.

Benefits:

- faster analytics,
- precomputed aggregations.

27. Indexes in SQL

Indexes improve query performance.

Create Index

```
CREATE INDEX idx_customer_city
ON customers(city);
```


How Indexes Work

Indexes act like book indexes.

Without index:

Full table scan

With index:

Fast lookup

Types of Indexes

Index Type	Description
Clustered	Physical order
Non-Clustered	Separate structure
Composite	Multiple columns
Unique	No duplicate values

When to Use Indexes

Good for:

- WHERE filters,
- JOIN columns,
- ORDER BY columns.

Avoid excessive indexes on:

- frequently updated tables.

28. Common Table Expressions (CTEs)

CTEs improve query readability.

Example

```
WITH revenue_cte AS (  
    SELECT customer_id,  
           SUM(amount) AS total_revenue  
    FROM orders  
    GROUP BY customer_id  
)  
SELECT *  
FROM revenue_cte;
```

Advantages of CTEs

- modular queries,
- easier debugging,
- reusable logic.

29. Recursive CTEs

Used for hierarchical data.

Example — Employee Hierarchy

```
WITH RECURSIVE employee_hierarchy AS (  
    SELECT employee_id,  
           employee_name,  
           manager_id  
    FROM employees  
    WHERE manager_id IS NULL  
  
    UNION ALL  
  
    SELECT e.employee_id,  
           e.employee_name,  
           e.manager_id  
    FROM employees e  
    JOIN employee_hierarchy eh  
    ON e.manager_id = eh.employee_id  
)  
SELECT *  
FROM employee_hierarchy;
```

30. Window Functions Deep Dive

Window functions perform calculations across rows.

Unlike GROUP BY:

- rows are not collapsed.

Window Function Syntax

```
FUNCTION() OVER (  
    PARTITION BY column  
    ORDER BY column  
)
```

31. Ranking Functions

ROW_NUMBER()

```
SELECT customer_id,  
       amount,  
       ROW_NUMBER() OVER (  
           ORDER BY amount DESC  
       ) AS row_num  
FROM orders;
```

RANK()

```
SELECT customer_id,  
       amount,  
       RANK() OVER (  
           ORDER BY amount DESC  
       ) AS ranking  
FROM orders;
```

DENSE_RANK()

```
SELECT customer_id,  
       amount,  
       DENSE_RANK() OVER (  
         ORDER BY amount DESC  
       ) AS dense_rank  
FROM orders;
```

Ranking Comparison

Function	Duplicate Handling
ROW_NUMBER	Unique sequence
RANK	Skips ranks
DENSE_RANK	No skipped ranks

32. Aggregate Window Functions

SUM() OVER()

```
SELECT customer_id,  
       amount,  
       SUM(amount) OVER () AS total_sales  
FROM orders;
```

AVG() OVER()

```
SELECT customer_id,  
       amount,  
       AVG(amount) OVER () AS avg_sales  
FROM orders;
```

33. Running Totals and Moving Averages

Running Total

```
SELECT order_date,
       amount,
       SUM(amount) OVER (
         ORDER BY order_date
       ) AS running_total
FROM orders;
```

Moving Average

```
SELECT order_date,
       amount,
       AVG(amount) OVER (
         ORDER BY order_date
         ROWS BETWEEN 2 PRECEDING AND CURRENT ROW
       ) AS moving_avg
FROM orders;
```

34. CASE Statements

Used for conditional logic.

Example

```
SELECT customer_name,
       amount,
       CASE
         WHEN amount > 500 THEN 'High'
         WHEN amount > 200 THEN 'Medium'
         ELSE 'Low'
       END AS order_category
FROM orders;
```

35. Set Operators

Operator	Description
UNION	Combine unique rows
UNION ALL	Combine all rows
INTERSECT	Common rows
EXCEPT	Difference

UNION Example

```
SELECT city FROM customers
UNION
SELECT city FROM suppliers;
```

36. Stored Procedures and Functions

Reusable database programs.

Stored Procedure Example

```
CREATE PROCEDURE GetCustomers()
BEGIN
    SELECT * FROM customers;
END;
```

Function Example

```
CREATE FUNCTION CalculateTax(amount DECIMAL(10,2))
RETURNS DECIMAL(10,2)
BEGIN
    RETURN amount * 0.10;
END;
```

37. Transactions and ACID Properties

Transactions ensure reliability.

ACID Properties

Property	Description
Atomicity	All or nothing
Consistency	Valid state
Isolation	Independent transactions
Durability	Permanent storage

38. SQL Performance Optimization

Best Practices

Use Proper Indexes

Improves lookup speed.

Avoid SELECT *

Retrieve only required columns.

Filter Early

Use WHERE clause effectively.

Optimize Joins

Join indexed columns.

Use Partitioning

Improves large-table performance.

Query Optimization Example

Bad:

```
SELECT *  
FROM huge_table;
```

Good:

```
SELECT customer_id,  
       amount  
FROM huge_table  
WHERE order_date >= '2026-01-01';
```

39. Real-World SQL Scenarios

E-Commerce Revenue Analysis

```
SELECT product_category,  
       SUM(revenue) AS total_revenue  
FROM fact_sales  
GROUP BY product_category;
```

Customer Retention Analysis

```
SELECT customer_id,  
       COUNT(order_id) AS order_count  
FROM orders  
GROUP BY customer_id;
```

Fraud Detection Example

```
SELECT account_id,  
       COUNT(*) AS transaction_count  
FROM transactions
```



```
WHERE transaction_amount > 10000  
GROUP BY account_id;
```

40. SQL Best Practices

Use Meaningful Names

Good:

```
customer_id  
order_amount
```

Bad:

```
cid  
amt
```

Format Queries Properly

Improves readability.

Avoid Duplicate Logic

Centralize reusable SQL.

Use Aliases Carefully

Improves clarity.

Write Modular Queries

Use CTEs and views.

41. Common SQL Mistakes

Missing WHERE Clause

```
DELETE FROM customers;
```

Deletes entire table data.

Incorrect JOIN Conditions

Creates duplicate rows.

Overusing SELECT *

Reduces performance.

Ignoring Indexes

Causes slow queries.

NULL Handling Errors

```
WHERE city = NULL
```

Correct:

```
WHERE city IS NULL
```

42. SQL Interview-Level Concepts

Every SQL engineer should understand:

Concept	Importance
Joins	Extremely High
GROUP BY	High
Window Functions	Extremely High
Indexes	High
CTEs	High
Transactions	High
Query Optimization	Extremely High
Subqueries	High
ACID	High

43. End-to-End SQL Project Example

Scenario

Build a sales reporting system.

Step 1 — Create Tables

```
CREATE TABLE customers (  
  customer_id INT,  
  customer_name VARCHAR(100)  
);
```

```
CREATE TABLE orders (  
  order_id INT,  
  customer_id INT,  
  amount DECIMAL(10,2)  
);
```

Step 2 — Insert Data

```
INSERT INTO customers  
VALUES (101, 'John');
```

```
INSERT INTO orders  
VALUES (1, 101, 500);
```

Step 3 — Build Analytical Query

```
SELECT c.customer_name,  
       SUM(o.amount) AS total_revenue  
FROM customers c  
JOIN orders o  
ON c.customer_id = o.customer_id  
GROUP BY c.customer_name;
```

Step 4 — Create View

```
CREATE VIEW customer_revenue AS  
SELECT c.customer_name,  
       SUM(o.amount) AS total_revenue  
FROM customers c  
JOIN orders o  
ON c.customer_id = o.customer_id  
GROUP BY c.customer_name;
```

Step 5 — Optimize with Index

```
CREATE INDEX idx_orders_customer  
ON orders(customer_id);
```

44. Final Summary

SQL is one of the most important skills in:

- Data Engineering,
- Analytics,
- Software Engineering,
- Business Intelligence,
- Database Administration.

Mastering SQL requires understanding:

- basic syntax,
- joins,
- subqueries,
- indexes,
- views,
- CTEs,
- window functions,
- transactions,
- performance optimization.

Modern enterprise SQL is used for:

`Analytics + Reporting + ETL + Warehousing + Real-Time Processing`

A strong SQL foundation enables:

- efficient data analysis,
- scalable applications,
- optimized queries,
- enterprise-grade reporting,
- reliable business insights.

SQL remains one of the most valuable technical skills across the entire data industry.